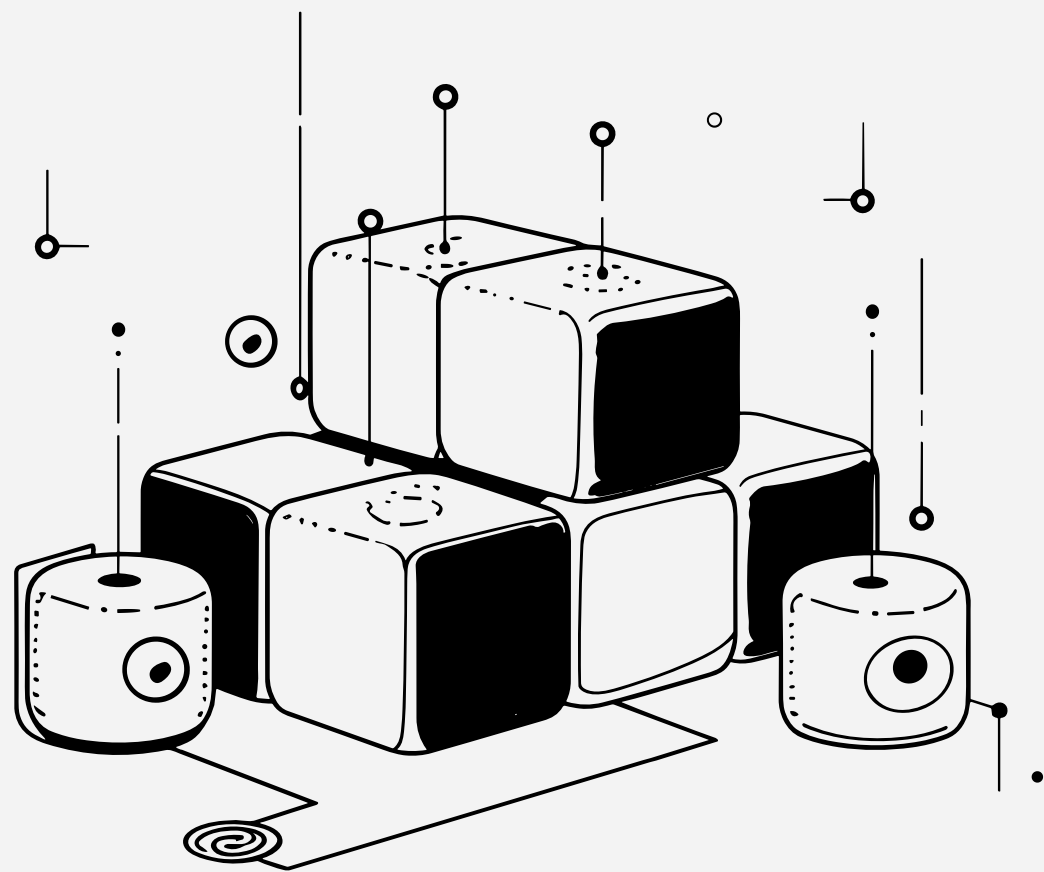




Objetos em JavaScript

Técnico em Desenvolvimento de Sistemas

Unidade Curricular: Programação Front-end

Profº Allyson Souza

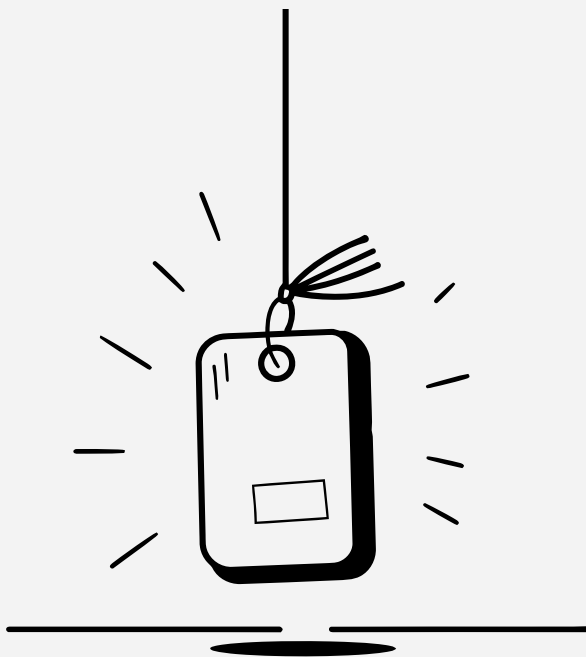


O que são objetos

Um objeto é um valor composto: ele agrega vários valores (valores primitivos ou outros objetos) e permite armazenar e recuperar esses valores pelo nome.

Um objeto é uma coleção **não ordenada** de propriedades, cada uma das quais possui um **nome** e um **valor**.

Propriedades



Os nomes das propriedades são geralmente cadeias de caracteres (embora os nomes das propriedades também possam ser símbolos); portanto, podemos dizer que os objetos mapeiam cadeias de caracteres para valores.

Propriedades

Nome da Propriedade

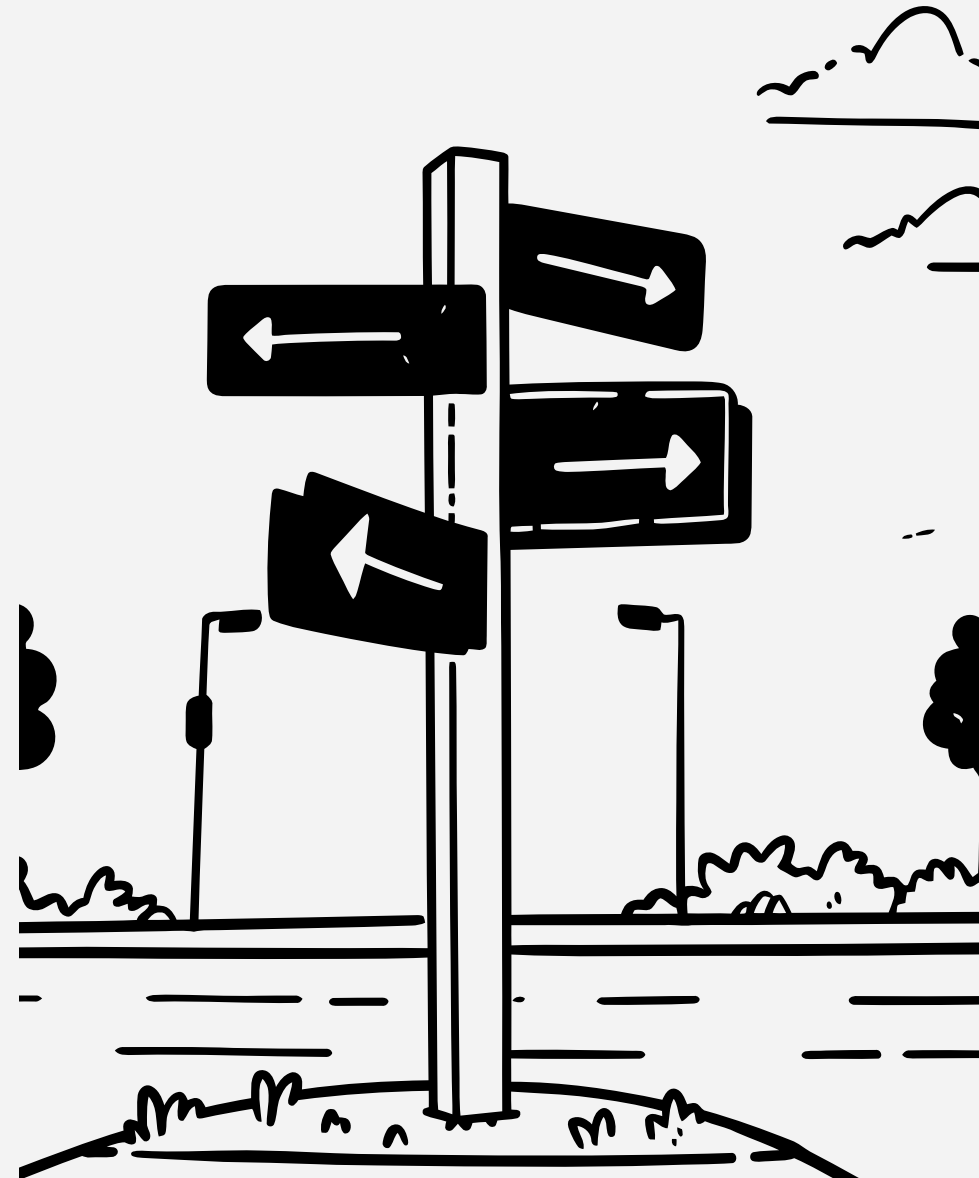
Pode ser qualquer sequência de caracteres, incluindo a sequência vazia (ou qualquer símbolo), mas nenhum objeto pode ter duas propriedades com o mesmo nome.

Valor da Propriedade

Qualquer valor em JavaScript que não seja uma string, um número, um Symbol ou `true`, `false`, `null` ou `undefined` é um objeto.

Criando objetos

Objetos podem ser criados utilizando objetos literais, o operador `new` ou com o método `Object.create()`. Iremos aprender cada uma das formas a seguir.



Criando objetos literais

O jeito mais simples de criação de objetos. De forma simples e resumida, um objeto literal (*object literals*) é uma lista separada por vírgulas (,) de nomes e valores, separados por dois pontos (:), envolvidos em chaves ({ }).

O valor de uma propriedade pode ser qualquer expressão ou valor válido em JavaScript.

```
let vazio = {}; // Um objeto sem propriedades
let ponto = { x: 0, y: 0 }; // Duas propriedades numéricas, x e y
let ponto2 = { x: point.x, y: point.y + 1 }; // Propriedades com valores mais complexos
let book = {
  "titulo": "JavaScript", // These property names include spaces,
  "sub-titulo": "O Guia Definitivo", // and hyphens, so use string literals.
  string: "todos os públicos", // string é uma palavra reservada, mas sem aspas pode ser usada como propriedade
  autor: { // O valor da propriedade autor
    nome: "David", // é também um objeto.
    sobrenome: "Flanagan"
  }
};
```

Criando objetos com new

O operador `new` cria e inicializa um novo objeto. A palavra-chave `new` deve ser seguida por uma chamada de função. Uma função usada dessa maneira é chamada de **construtor** e serve para inicializar um objeto recém-criado. O JavaScript inclui construtores para seus tipos embutidos.

Em adição aos construtores internos da linguagem, é comum que definamos as nossas próprias funções construtoras para criação de novos objetos.

```
let o = new Object(); // Cria um objeto vazio: mesmo  
que {}.  
let a = new Array(); // Cria um objeto vazio: mesmo  
que [].  
let d = new Date(); // Cria um objeto Date  
representando o horário atual  
let r = new Map(); // Cria um objeto Map para  
mapeamento de chave/valor
```

Criando objetos com `Object.create()`

`Object.create()` cria um novo objeto usando o seu primeiro argumento como o protótipo do objeto a ser criado. Exemplo:

```
let objeto1 = Object.create({x: 1, y: 2}); // objeto1 herda as propriedades x e y.  
objeto1.x + objeto1.y // resultado: 3
```

- ❏ A criação de objetos a partir de protótipos arbitrários é um recurso poderoso da linguagem que exploraremos mais adiante.

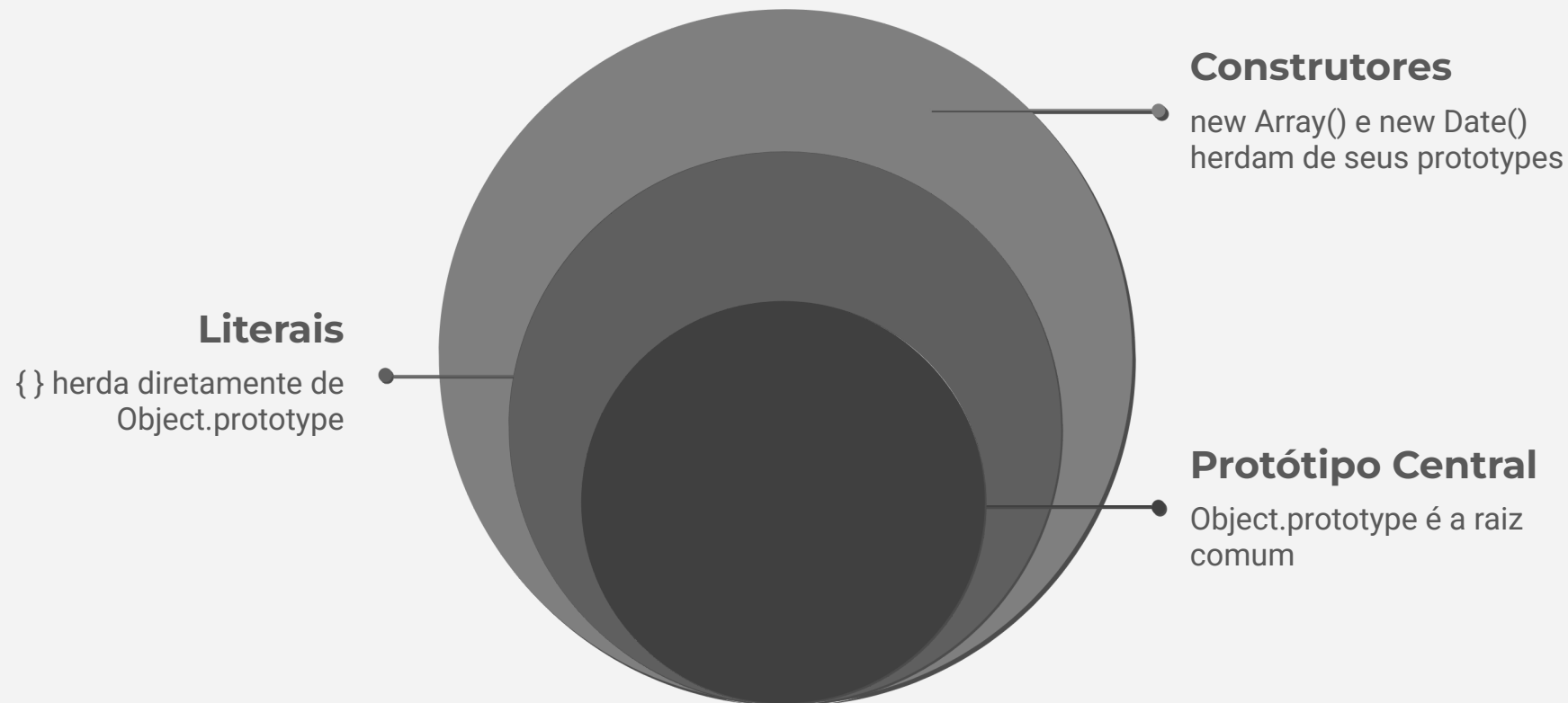


Protótipos

Um objeto é mais do que um simples mapeamento de string para valor, no entanto. Além de manter seu próprio conjunto de propriedades, um objeto JavaScript também herda as propriedades de outro objeto, conhecido como seu "protótipo".

Os métodos de um objeto são tipicamente propriedades herdadas, e essa "herança prototípica" é uma característica fundamental do JavaScript, que iremos cobrir mais adiante.

Protótipos



Todos os objetos criados literalmente têm o mesmo objeto protótipo: `Object.prototype`. Os objetos criados com `new` utilizam o valor da propriedade `prototype` da função construtora como seu protótipo. Assim, `new Array()` usa `Array.prototype` e `new Date()` usa `Date.prototype`.

Protótipos

⚠ **Atenção:** isso pode ser confuso quando se está começando a aprender JavaScript. Lembre-se: quase todos os objetos têm um protótipo, mas apenas um número relativamente pequeno de objetos possui uma propriedade `prototype`. São esses objetos com propriedades `prototype` que definem os protótipos para todos os outros objetos.

Protótipos — Cadeia de Protótipos

`Object.prototype` é um dos raros objetos que não possui protótipo: ele não herda nenhuma propriedade. Outros objetos protótipos são objetos normais que possuem um protótipo. A maioria dos construtores embutidos (e a maioria dos construtores definidos pelo usuário) possui um protótipo que herda de `Object.prototype`.

Por exemplo, `Date.prototype` herda propriedades de `Object.prototype`; portanto, um objeto `Date` criado por `new Date()` herda propriedades tanto de `Date.prototype` quanto de `Object.prototype`. Essa série interligada de objetos protótipos é conhecida como cadeia de protótipos (*prototype chain*).

Acessando valores de propriedades

Para acessar propriedades de um objeto, nós usamos o operador `.` ou `[]`. À esquerda precisamos de um identificador que é um objeto, à direita do ponto ou dentro dos colchetes, entre aspas, o nome do identificador de uma propriedade existente no objeto.

```
const livro = {  
  "titulo": "Dom Casmurro",  
  "autor": "Machado de Assis",  
  "número ISBN": "9794712283172"  
}
```

```
console.log( livro.titulo ); // notação de ponto  
console.log( livro.autor ); // notação de ponto  
console.log( livro["número ISBN"] ); // Utilizando [], já que o nome da propriedade possui espaços
```

Acessando uma propriedade que não existe

Tentar acessar uma propriedade que não existe, irá retornar o valor `undefined`.

```
livro.edicao // undefined
```

```
livro.ano // undefined
```

Atribuindo valores

Podemos atribuir novos valores mutando nossos objetos, basta utilizarmos o operador de `=` à direita de uma expressão que seleciona uma propriedade de um objeto (com `.` ou `[]`), exemplo:

```
livro.titulo = "Recordações do Escrivão Isaías Caminha"  
livro.autor = "Lima Barreto"  
livro["número ISBN"] = "98737412358163"
```

Apagando propriedades

O operador `delete` remove uma propriedade de um objeto. Seu único operando deve ser uma expressão de acesso à propriedade. Surpreendentemente, o `delete` não atua sobre o valor da propriedade, mas sobre a própria propriedade:

```
delete livro.autor;  
delete livro["número ISBN"];  
// Após a execução destas linhas, o objeto  
// não terá mais estas propriedades definidas
```

Verificar se objeto tem propriedade

Muitas vezes iremos lidar com objetos que foram previamente definidos, ou que em determinado trecho de código não temos certeza se uma propriedade existe ou não. Para verificar se um objeto tem uma propriedade, podemos utilizar `hasOwn()` ou o operador `in`.

Object.hasOwn()

Verifica apenas propriedades próprias do objeto

Operador `in`

Verifica propriedades próprias e herdadas via prototype chain



hasOwn

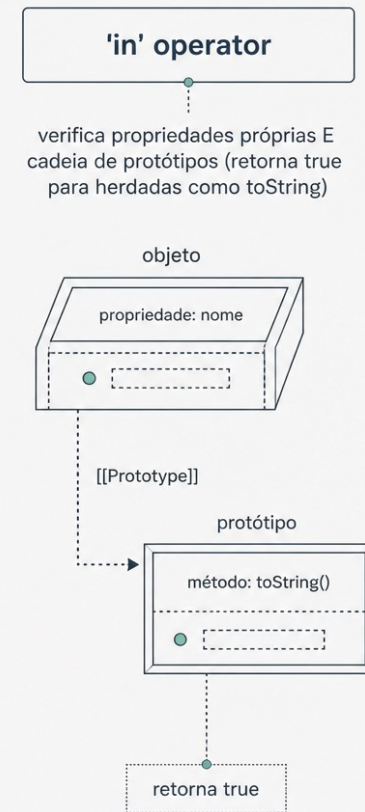
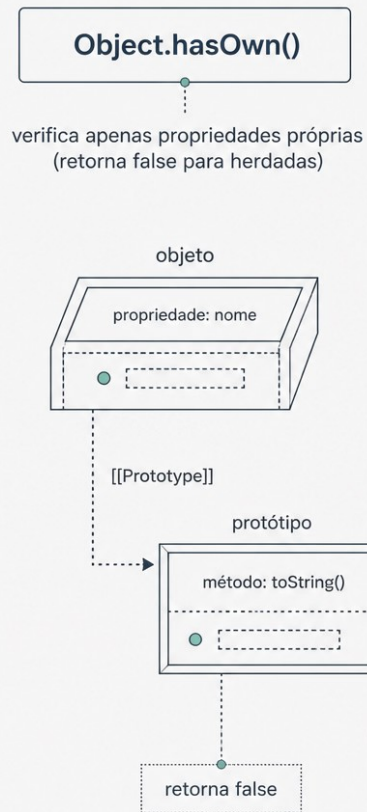
O método `Object.hasOwn()` retorna `true` se o objeto especificado tem a propriedade indicada como sua própria propriedade. Se a propriedade é herdada, ou não existe, o método retorna `false`.

```
const carro = {  
  cor: "vermelho",  
};  
  
console.log(Object.hasOwn(carro, "cor")); // true  
  
console.log(Object.hasOwn(carro, "toString")); // false - ele tem a propriedade toString,  
// mas ela foi herdada do protótipo Object  
  
console.log(Object.hasOwn(carro, "modelo")); // false
```

Operador in

O operador `in` retorna `true` se a propriedade especificada estiver no objeto especificado **ou** na sua cadeia de protótipo (*prototype chain*) desde `Object`.

```
const carro = {  
  cor: "vermelho",  
};  
  
"cor" in carro;    // true  
"toString" in carro; // true  
"modelo" in carro; // false
```





Métodos

Todos os objetos JavaScript (exceto aqueles criados explicitamente sem um protótipo) herdam propriedades de `Object.prototype`. Essas propriedades herdadas são, principalmente, métodos e, por estarem universalmente disponíveis, são de particular interesse para os programadores de JavaScript.

Mas também podemos definir nossos próprios métodos, definindo funções em nossos objetos.

Declaração de método em objeto literal

Para declararmos métodos em objetos literais, podemos utilizar a sintaxe abreviada, declarando um método com seu nome, lista de parâmetros e corpo, dentro das {} que delimitam o objeto:

```
const samurai = {  
  nome: "Musashi",  
  oficio: "espadachim",  
  estilo: "Niten Ichi-Ryu",  
  
  apresentar() {  
    return 'Olá, meu nome é Musashi, sou espadachim';  
  }  
};  
  
console.log(samurai.apresentar());
```

- ❏ Perceba que o método não está utilizando as propriedades do objeto para construir a mensagem retornada. E se o valor de nome e oficio mudarem? Precisaremos alterar também no método. Podemos resolver isso com a palavra reservada `this`.

this

A palavra reservada `this` tem seu valor definido a partir do contexto em que ela é utilizada e em que uma função ou método é invocado. Vamos adaptar o exemplo anterior para, usando `this`, acessar as propriedades do objeto.

```
const samurai = {  
  nome: "Musashi",  
  oficio: "espadachim",  
  estilo: "Niten Ichi-Ryu",  
  
  apresentar() {  
    return `Olá, meu nome é ${this.nome}, sou ${this.oficio}`;  
  }  
};  
  
console.log(samurai.apresentar());
```

Quando o método `apresentar()` é invocado, o valor de `this` estará referenciando o próprio objeto, sendo assim possível acessar as propriedades dele de dentro do método. Esse é um recurso bastante poderoso que nos permite criar objetos ainda mais complexos.

Obs: no contexto de execução global (fora de qualquer função), `this` refere-se ao objeto global.

this e escopo

A regra de ouro para entender o `this` no JavaScript se resume a uma diferença simples de conceito:

Funções normais (`function` ou métodos tradicionais)

O `this` é **dinâmico** — depende de como e por quem a função é invocada no momento da execução.

Arrow Functions (`=>`)

O `this` é **léxico** (estático) — ela não possui seu próprio `this` e apenas herda o valor do escopo externo onde foi definida.

Iremos estudar em mais detalhes o comportamento do `this` de forma mais aprofundada mais adiante.

Referências

- FLANAGAN, David. **JavaScript: The Definitive Guide**. 7. ed. Sebastopol: O'Reilly Media, 2020. p. 129-153.
- Mozilla Developer Network. **Object.hasOwn()**. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/hasOwn. Acessado em: 05 de agosto de 2026.
- _____. **this**. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/this>. Acessado em: 05 de agosto de 2026.